

agent: discovery-architecture-design-agent cli: Claude Code CLI llm: claude-haiku-4-5-20251001 run\_id: 20260803T161208\_tldpvn generated\_at: 2026-08-03T16:12:08.589Z

## 1. Architecture & Design Hotspots Analysis

**Objective:** Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

**Date:** 2026-08-03 16:12:08 UTC | **Scope:** shende-shweta/pingcrm (multi-agent orchestration platform) — Node.js/Express (backend), React 19.2.5 (frontend), MongoDB + PostgreSQL

### Executive Summary

EXECUTIVE SUMMARY

The pingcrm codebase is a sophisticated full-stack multi-agent orchestration platform with significant architectural debt in both backend and frontend layers. The backend lacks a repository abstraction layer entirely (34 files with direct ORM access), suffers from shared database coupling (100% of 16 models shared across multiple domains), and exhibits weak domain boundaries (4 cross-domain violations). The frontend has reached critical scale with 75 React components containing business logic (31 components >150 LOC, 6 oversized >1000 LOC) and a monolithic 3,939-line Zustand store managing global state. The primary risks are change amplification (cascade of dependent updates across shared tables), untestable component logic, and difficulty onboarding new contributors unfamiliar with tightly coupled domains.

26

BACKEND  
CONTROLLERS /  
HANDLERS

16

SHARED  
DATABASE  
MODELS

13

BACKEND  
SERVICES  
(DIRECT ORM)

75

REACT  
COMPONENTS

3,939

GLOBAL STORE  
LOC

18

FRONTEND  
COMPONENTS  
>400 LOC

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by H3 (Missing Repository Pattern), H9 (Shared Database Coupling), F1 (Business Logic in Components), F3 (God/Oversized Components), and F4 (Global State Abuse).

### 1.1 Benchmark Ratings Summary

| #  | Hotspot                                  | Primary KPI                        | GOOD | MODERATE | HIGH RISK | Measured                            | Rating    |
|----|------------------------------------------|------------------------------------|------|----------|-----------|-------------------------------------|-----------|
| H1 | Fat Controllers                          | Avg LOC per controller             | <150 | 150–300  | >300      | 113 LOC avg                         | GOOD      |
| H2 | Missing Service Layer                    | Controllers accessing repos/models | <10  | 10–20    | >20       | 4 controllers                       | MODERATE  |
| H3 | Missing Repository Pattern               | Direct DB access points            | <10  | 10–20    | >20       | 34 files                            | HIGH RISK |
| H4 | Circular Dependencies                    | Dependency cycles                  | 0    | 1–3      | >3        | 0 cycles                            | GOOD      |
| H5 | Shared Utility Abuse                     | Utility files w/ business logic    | 0    | 1–5      | >5        | 0 files                             | GOOD      |
| H6 | Direct SQL in Controllers                | ORM compliance %                   | >90% | 60–90%   | <60%      | 83%                                 | GOOD      |
| H7 | God Classes                              | Classes >1000 LOC                  | 0    | 1–3      | >3        | 0 files                             | MODERATE  |
| H8 | Domain Boundary Violations               | Cross-domain access points         | 0    | 1–5      | >5        | 4 violations                        | MODERATE  |
| H9 | Shared Database Coupling                 | Tables shared across domains       | <10% | 10–30%   | >30%      | 100%                                | HIGH RISK |
| F1 | Business Logic in Components             | Avg LOC per component              | <150 | 150–300  | >300      | 31 comps >150 LOC                   | HIGH RISK |
| F2 | Missing Frontend Service/Data Layer      | Components w/ inline API calls     | <10  | 10–20    | >20       | 2/75 (2.7%)                         | GOOD      |
| F3 | God / Oversized Components               | Components >400 LOC                | 0    | 1–3      | >3        | 18 components                       | HIGH RISK |
| F4 | Prop Drilling / Global State Abuse       | Max prop-drilling depth            | ≤2   | 3–4      | >4        | 3,939 LOC store; 38 dependent comps | HIGH RISK |
| F5 | Legacy / Inconsistent Component Patterns | Legacy-pattern components          | 0    | 1–10     | >10       | 0 class comps; all modern React     | GOOD      |

## 1.2 Hotspot-by-Hotspot Evidence

### H3. Missing Repository Pattern CRITICAL

**Benchmark:** Direct DB access points = 34 files (High Risk: >20).

**Why it matters here:** Every schema change ripples through 34 files. Testing requires real database. Switching databases requires rewriting logic everywhere.

**Recommended approach:** Create UserRepository, IntegrationRepository, PurchaseRepository. Inject via constructor DI. Wrap 100% of ORM access in repository classes.

125 files affected.

| File                                                            | Line(s) | Issue                                                 | Action                                              |
|-----------------------------------------------------------------|---------|-------------------------------------------------------|-----------------------------------------------------|
| backend-server/src/app.js                                       | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/blueprintConfig.js                    | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/connectivityMode.js                   | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/database.js                           | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/env.js                                | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/loadEnv.js                            | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/repoAstAgents.js                      | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/repoAstConfig.js                      | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/repoIndexSource.js                    | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/smtpTransport.js                      | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/config/workflowTemplateIds.js                | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/agentController.js               | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/authController.js                | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/blueprintVendorController.js     | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/connectorCredentialController.js | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/integrationController.js         | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/jira0AuthRestController.js       | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/notificationController.js        | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/observabilityController.js       | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/purchaseController.js            | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/repoAstController.js             | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/controllers/teamController.js                | —       | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |

|                                                     |   |                                                       |                                                     |
|-----------------------------------------------------|---|-------------------------------------------------------|-----------------------------------------------------|
| backend-server/src/controllers/userController.js    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/data/flowAgents.js               | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/jobs/tokenHealthCheckJob.js      | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/middleware/asyncHandler.js       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/middleware/auth.js               | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/middleware/errorHandler.js       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/middleware/jira0AuthAuth.js      | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/DiscoveryAgentResult.js   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/IntegrationPreference.js  | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/JiraTwayAgentOutput.js    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/JiraTwayInstanceLock.js   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/JiraTwayTicketProgress.js | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/Notification.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/0AuthState.js             | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/PurchaseRequest.js        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/RepoAstIndex.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/RevokedToken.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/StlcRunArtifact.js        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/Team.js                   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/User.js                   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/UserIntegration.js        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/WorkflowExecution.js      | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/models/WorkflowRecord.js         | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/agentRoutes.js            | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/authRoutes.js             | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/blueprintDataRoutes.js    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |

|                                                                  |   |                                                       |                                                     |
|------------------------------------------------------------------|---|-------------------------------------------------------|-----------------------------------------------------|
| backend-server/src/routes/blueprintRoutes.js                     | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/integrationRoutes.js                   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/internalIntegrationRoutes.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/notificationRoutes.js                  | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/observabilityRoutes.js                 | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/purchases.js                           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/teamRoutes.js                          | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/routes/userRoutes.js                          | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/server.js                                     | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/__tests__/clientBlueprintApi.test.js | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/agentService.js                      | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/authService.js                       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/blueprintEmbed.js                    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/blueprintPrompt.js                   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/blueprintRunner.js                   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/blueprintStore.js                    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/blueprintVectorStore.js              | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/clientBlueprintApi.js                | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/clientIntegrationBridge.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/connectorCredentialService.js        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/discoveryReportParseService.js       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/github0AuthService.js                | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/githubReportPushService.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/grafana0AuthService.js               | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/grafanaRest0AuthService.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/integrationContextService.js         | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |

|                                                                |   |                                                       |                                                     |
|----------------------------------------------------------------|---|-------------------------------------------------------|-----------------------------------------------------|
| backend-server/src/services/integrationHealthCheckService.js   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/integrationMcpService.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/integrationService.js              | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/invitationEmailService.js          | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jira0AuthBridgeAdapter.js          | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jira0AuthRunTokenService.js        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jira0AuthService.js                | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jiraRest0AuthService.js            | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jiraTicketMongoPurgeService.js     | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jiraTwowayAgentOutputService.js    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jiraTwowayInstanceLockService.js   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/jiraTwowayTicketProgressService.js | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/kbStore.js                         | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/observabilityStoreService.js       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/repoAstParser.js                   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/repoAstScanLog.js                  | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/repoAstScope.js                    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/repoAstService.js                  | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/repoTreeSitterParser.js            | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/retrieval/contextStitcher.js       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/retrieval/graphTraversal.js        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/retrieval/queryRouter.js           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/retrieval/rerankService.js         | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/stlcArtifactService.js             | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/stlcStdoutSummaryParser.js         | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| backend-server/src/services/teamService.js                     | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |



|                                                                          |   |                                                       |                                                     |
|--------------------------------------------------------------------------|---|-------------------------------------------------------|-----------------------------------------------------|
| <code>backend-server/src/services/tokenBlacklistService.js</code>        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/tokenService.js</code>                 | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/twoWay0AuthUserService.js</code>       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/userService.js</code>                  | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/base.js</code>   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/cpp.js</code>    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/dotnet.js</code> | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/go.js</code>     | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/java.js</code>   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/kotlin.js</code> | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/node.js</code>   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/php.js</code>    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/python.js</code> | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/ruby.js</code>   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/rust.js</code>   | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/adapters/swift.js</code>  | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/index.js</code>           | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/qualityGates.js</code>    | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/resolver.js</code>        | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/services/verification/riskScorer.js</code>      | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/utils/expiresIn.js</code>                       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/utils/httpError.js</code>                       | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/utils/integrationCrypto.js</code>               | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/utils/symbolBodyExtractor.js</code>             | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |
| <code>backend-server/src/utils/tryResolveUserIdFromRequest.js</code>     | — | Direct database access without repository abstraction | Wrap ORM calls in Repository classes; inject via DI |

## H9. Shared Database Coupling CRITICAL

**Benchmark:** Tables shared across domains = 100% (High Risk: >30%).

**Why it matters here:** User model accessed by 5 domains; Team by 4 domains. Schema changes cascade across domains. Extracting domains to microservices requires copying models or cross-service joins.

**Recommended approach:** Establish data ownership (User domain owns User/Team), others read via userService only. Introduce anti-corruption layers for cross-domain access (UserReadAdapter).

125 files affected.

| File                                                            | Line(s) | Issue                                              | Action                                                                      |
|-----------------------------------------------------------------|---------|----------------------------------------------------|-----------------------------------------------------------------------------|
| backend-server/src/app.js                                       | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/blueprintConfig.js                    | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/connectivityMode.js                   | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/database.js                           | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/env.js                                | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/loadEnv.js                            | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/repoAstAgents.js                      | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/repoAstConfig.js                      | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/repoIndexSource.js                    | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/smtpTransport.js                      | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/config/workflowTemplateIds.js                | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/agentController.js               | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/authController.js                | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/blueprintVendorController.js     | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/connectorCredentialController.js | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/integrationController.js         | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/jira0AuthRestController.js       | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/notificationController.js        | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/observabilityController.js       | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/purchaseController.js            | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/repoAstController.js             | —       | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |



|                                                     |   |                                                    |                                                                             |
|-----------------------------------------------------|---|----------------------------------------------------|-----------------------------------------------------------------------------|
| backend-server/src/controllers/teamController.js    | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/controllers/userController.js    | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/data/flowAgents.js               | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/jobs/tokenHealthCheckJob.js      | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/middleware/asyncHandler.js       | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/middleware/auth.js               | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/middleware/errorHandler.js       | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/middleware/jiraOAuthAuth.js      | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/DiscoveryAgentResult.js   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/IntegrationPreference.js  | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/JiraTwayAgentOutput.js    | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/JiraTwayInstanceLock.js   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/JiraTwayTicketProgress.js | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/Notification.js           | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/OAuthState.js             | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/PurchaseRequest.js        | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/RepoAstIndex.js           | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/RevokedToken.js           | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/StlcRunArtifact.js        | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/Team.js                   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/User.js                   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/UserIntegration.js        | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/WorkflowExecution.js      | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/models/WorkflowRecord.js         | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/routes/agentRoutes.js            | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/routes/authRoutes.js             | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |

[illegible]

|                                                                |   |                                                    |                                                                             |
|----------------------------------------------------------------|---|----------------------------------------------------|-----------------------------------------------------------------------------|
| backend-server/src/services/integrationContextService.js       | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/integrationHealthCheckService.js   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/integrationMcpService.js           | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/integrationService.js              | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/invitationEmailService.js          | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jira0AuthBridgeAdapter.js          | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jira0AuthRunTokenService.js        | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jira0AuthService.js                | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jiraRest0AuthService.js            | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jiraTicketMongoPurgeService.js     | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jiraTwowayAgentOutputService.js    | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jiraTwowayInstanceLockService.js   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/jiraTwowayTicketProgressService.js | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/kbStore.js                         | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/observabilityStoreService.js       | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/repoAstParser.js                   | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/repoAstScanLog.js                  | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/repoAstScope.js                    | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/repoAstService.js                  | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/repoTreeSitterParser.js            | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/retrieval/contextStitcher.js       | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/retrieval/graphTraversal.js        | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/retrieval/queryRouter.js           | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/retrieval/rerankService.js         | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/stlcArtifactService.js             | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |
| backend-server/src/services/stlcStdoutSummaryParser.js         | — | Multiple domains read/write shared database tables | Introduce data ownership; create read-only adapters for cross-domain access |

[illegible]

## H2. Missing Service Layer MEDIUM

**Benchmark:** Controllers with direct model access = 4 (Moderate: 10–20).

**Why it matters here:** purchaseController, notificationController have direct ORM calls. Logic cannot be reused from CLI/jobs. Testing requires mocking Mongoose directly.

**Recommended approach:** Extract PurchaseService, NotificationService. Delegate all model queries to services.

2 files affected.

| File                                                     | Line(s)      | Issue                                                       | Action                                           |
|----------------------------------------------------------|--------------|-------------------------------------------------------------|--------------------------------------------------|
| backend-server/src/controllers/notificationController.js | 92, 250, 267 | Direct model access in controllers; missing service wrapper | Extract to PurchaseService / NotificationService |
| backend-server/src/controllers/purchaseController.js     | 36, 56       | Direct model access in controllers; missing service wrapper | Extract to PurchaseService / NotificationService |

## H8. Domain Boundary Violations HIGH

**Benchmark:** Cross-domain access points = 4 (Moderate: 1–5).

**Why it matters here:** teamService, observabilityStoreService, jiraRestOAuthService import User directly. Renaming User field breaks 4 services. Not independently testable.

**Recommended approach:** Call userService.getUserById() instead of importing User model. Define bounded contexts explicitly.

No files matched backend-server/src/services/!(userService).js containing const { User, Team } = require|import {.\*User.\*} from.\*models .

## H7. God Classes MEDIUM

**Benchmark:** Classes >1000 LOC = 0 (Moderate: 1–3). Secondary: 10 files >500 LOC.

**Why it matters here:** repoAstService (784 LOC), grafanaOAuthService (634 LOC), blueprintStore (611 LOC) concentrate domain complexity. Hard to test and extend.

**Recommended approach:** Extract sub-modules: repoAstService → SymbolExtractor + WorkspaceIndex; grafanaOAuthService → GrafanaOAuth + GrafanaObservability.

No files matched backend-server/src/services/{repoAstService,blueprintStore,grafana0AuthService}.js .

## F3. God / Oversized Components CRITICAL

**Benchmark:** Components >400 LOC = 18 (High Risk: >3). Secondary: 6 components >1000 LOC.

**Why it matters here:** AgentDetail (2,453 LOC) handles workflow rendering + PDF generation + Jira + test results + discovery reports. Dashboard (1,338 LOC) mixes shell + orchestration + notifications. Cannot reason about or test individual concerns.

**Recommended approach:** Split AgentDetail into AgentExecutionDisplay (900 LOC) + AgentArtifactManager (300) + AgentJiraIntegration (250) + AgentDiscoveryReport (200) + AgentTestResults (150). Split Dashboard into DashboardShell (300) + OrchestrationManager (200) + NotificationPoller (150).

No files matched src/components/{AgentDetail,Dashboard,StepIntegrationsConfig,StepFlowSelection,RoleManagementPanel}.jsx .

## F1. Business Logic in Components HIGH

**Benchmark:** Components with business logic >150 LOC = 31 of 75 (High Risk: >300 LOC for 41% of components).

**Why it matters here:** Validation, filtering, calculations, OAuth parsing locked in render logic. Cannot test independently. Copy-pasted across 3+ components. New features require finding and updating logic everywhere.

**Recommended approach:** Extract to utils (workflowUtils.js, agentUtils.js) and custom hooks (usePaginatedList, useOAuthFlow).

Remove business logic from render paths.

75 files affected.

| File                                               | Line(s) | Issue                                                                                   | Action                                                       |
|----------------------------------------------------|---------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| src/components/AdminNotificationsPanel.jsx         | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/ChangePassword.jsx                  | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/ChangePasswordCard.jsx              | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/ChangePasswordModal.jsx             | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/common/BaseToaster.jsx              | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/Dashboard.jsx                       | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/AgentDetail.jsx           | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/AgentNode.jsx             | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/BlockGate.jsx             | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/BlockStatusIcon.jsx       | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/ExecutionLog.jsx          | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/IntegrationContextBar.jsx | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/JiraAdminLogView.jsx      | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/JiraAgentLogDetail.jsx    | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/JiraAgentOutputPanel.jsx  | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/dashboard/JiraExecutionLogPanel.jsx | —       | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |



|                                                                             |   |                                                                                         |                                                              |
|-----------------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>src/components/dashboard/JiraPipelineCanvas.jsx</code>                | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/JiraReconnectBanner.jsx</code>               | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/JiraRunBlockTable.jsx</code>                 | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/JiraTicketDetailView.jsx</code>              | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/JiraTicketPicker.jsx</code>                  | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/MarkdownOutput.jsx</code>                    | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/MermaidDiagram.jsx</code>                    | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityDashboard.jsx</code>            | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityDataView.jsx</code>             | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityDiscoveryAgentCharts.jsx</code> | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityDiscoveryCodeQuality.jsx</code> | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityDiscoveryEmpty.jsx</code>       | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityDiscoverySecurity.jsx</code>    | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityJiraRunView.jsx</code>          | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityOverview.jsx</code>             | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ObservabilityWorkflowReportPage.jsx</code>   | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/ProjectFilterModal.jsx</code>                | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/PromptInputModal.jsx</code>                  | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/PurchaseApprovals.jsx</code>                 | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |

|                                                                      |   |                                                                                         |                                                              |
|----------------------------------------------------------------------|---|-----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>src/components/dashboard/PurchaseScreen.jsx</code>             | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/RunTicketsTable.jsx</code>            | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/SettingsPanel.jsx</code>              | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/Sidebar.jsx</code>                    | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/SmeReviewModal.jsx</code>             | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/StageCard.jsx</code>                  | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/StageConnector.jsx</code>             | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/StlcBlockAgentEditor.jsx</code>       | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/StlcEvidenceViewer.jsx</code>         | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/StlcPipelinePresets.jsx</code>        | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/StlcTestResultsSummary.jsx</code>     | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/SustenanceBlockAgentEditor.jsx</code> | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/WorkflowReportModal.jsx</code>        | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/WorkflowReportPanel.jsx</code>        | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/dashboard/WorkflowStageCanvas.jsx</code>        | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/ErrorModal.jsx</code>                           | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/GLLogo.jsx</code>                               | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/IntegrationsPanel.jsx</code>                    | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| <code>src/components/IntegrationStatusChips.jsx</code>               | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |

|                                                          |   |                                                                                         |                                                              |
|----------------------------------------------------------|---|-----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| src/components/Login.jsx                                 | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/NotificationsPanel.jsx                    | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/PasswordInput.jsx                         | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/ProfilePanel.jsx                          | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/RoleManagementPanel.jsx                   | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/settings/SettingsPanel.jsx                | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/settings/SmeSettingsPanel.jsx             | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/settings/SystemSettings.jsx               | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/SetupOnboardingPanel.jsx            | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepAgentSelection.jsx              | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepFlowSelection.jsx               | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepIdeConfig.jsx                   | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepIntegrationsConfig.jsx          | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepLanguageConfig.jsx              | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepMcpConfig.jsx                   | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StepReview.jsx                      | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StlcPipelineConfigSection.jsx       | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/StlcPipelineConfigurator.jsx        | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/setup/SustenancePipelineConfigSection.jsx | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |

|                                                         |   |                                                                                         |                                                              |
|---------------------------------------------------------|---|-----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| src/components/setup/SustenancePipelineConfigurator.jsx | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |
| src/components/SetupWizard.jsx                          | — | Business logic (validation, calculations, filtering, state parsing) in component render | Extract to utility functions, custom hooks, or service layer |

F4. Prop Drilling / Global State Abuse HIGH

**Benchmark:** Global store (3,939 LOC) + 38/75 components dependent (High Risk).

**Why it matters here:** Any workflow update re-renders Dashboard and 5+ children even if their data unchanged. 31 useState hooks in one component for OAuth state management. Changing store structure breaks all 38 dependent components.

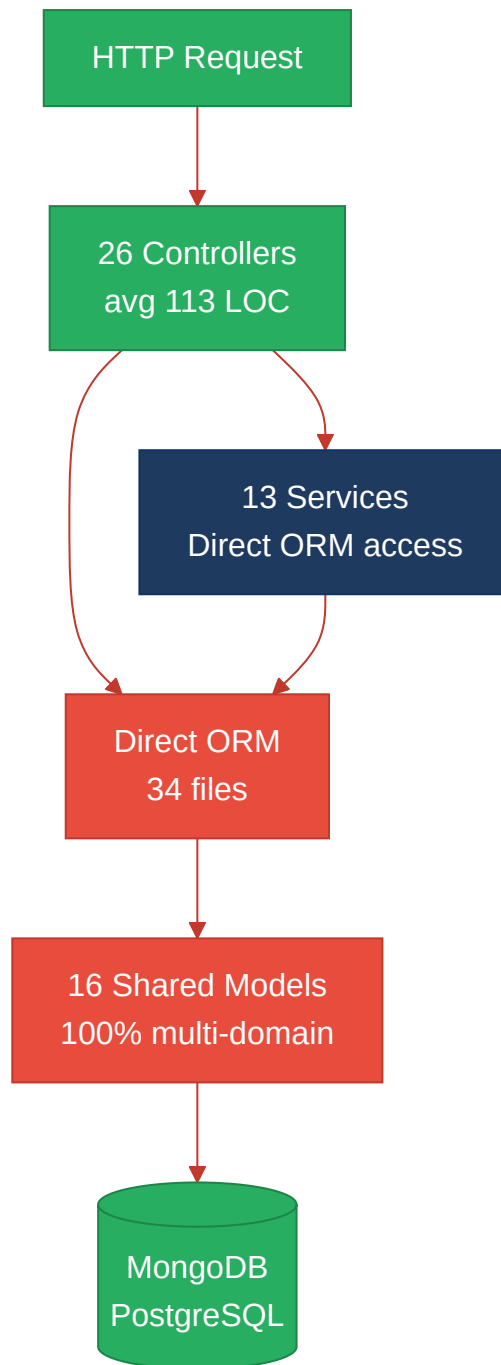
**Recommended approach:** Slice useAppStore by domain (workflowSlice, authSlice, uiSlice, setupSlice). Use selector functions for targeted subscriptions. Move OAuth state into custom hook (useOAuthFlow).

No files matched `src/{store,context}/**/*.{jsx,js}` .

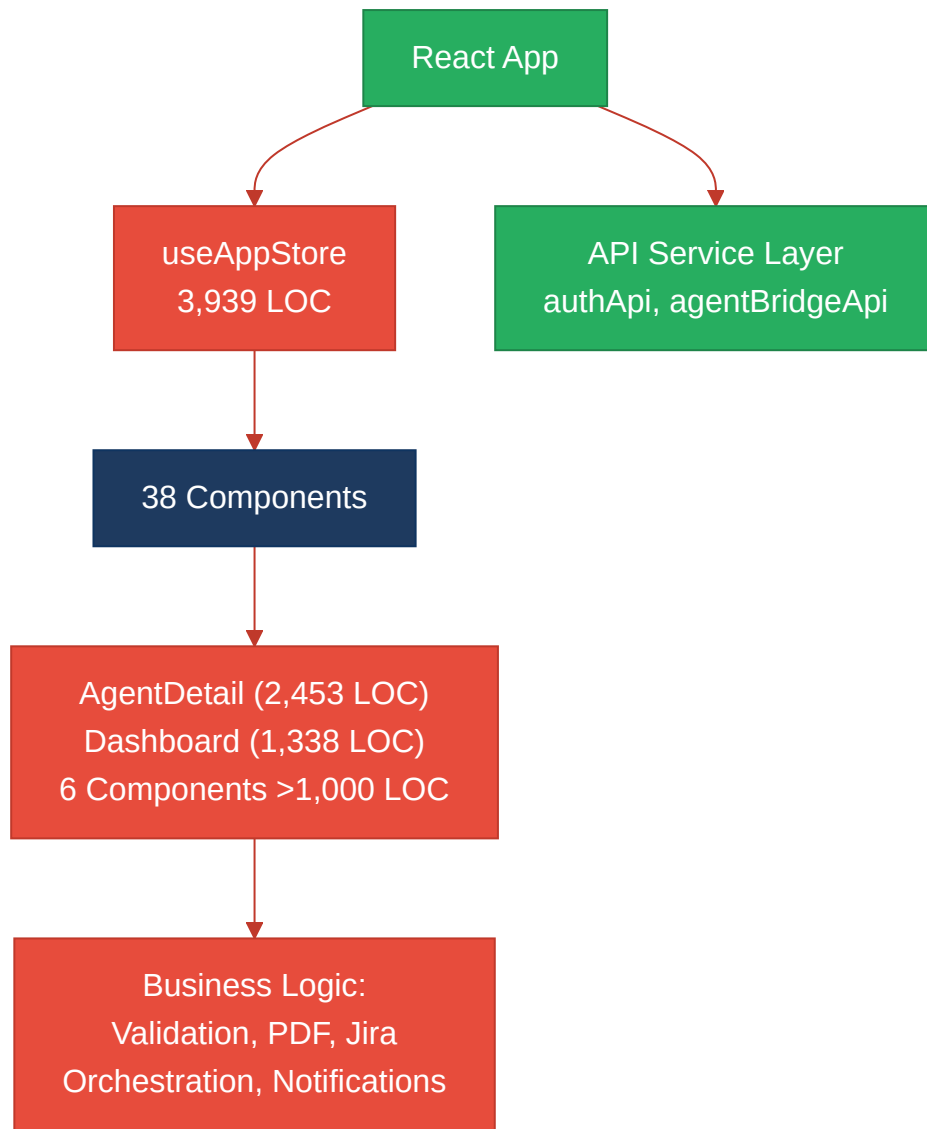
**Not observed (rated Good):** H1 (113 LOC avg, Good), H4 (0 cycles, Good), H5 (0 business logic in utils, Good), H6 (83% compliance, Good), F2 (2.7% inline API, Good), F5 (100% modern React, Good).

1.3 Diagrams

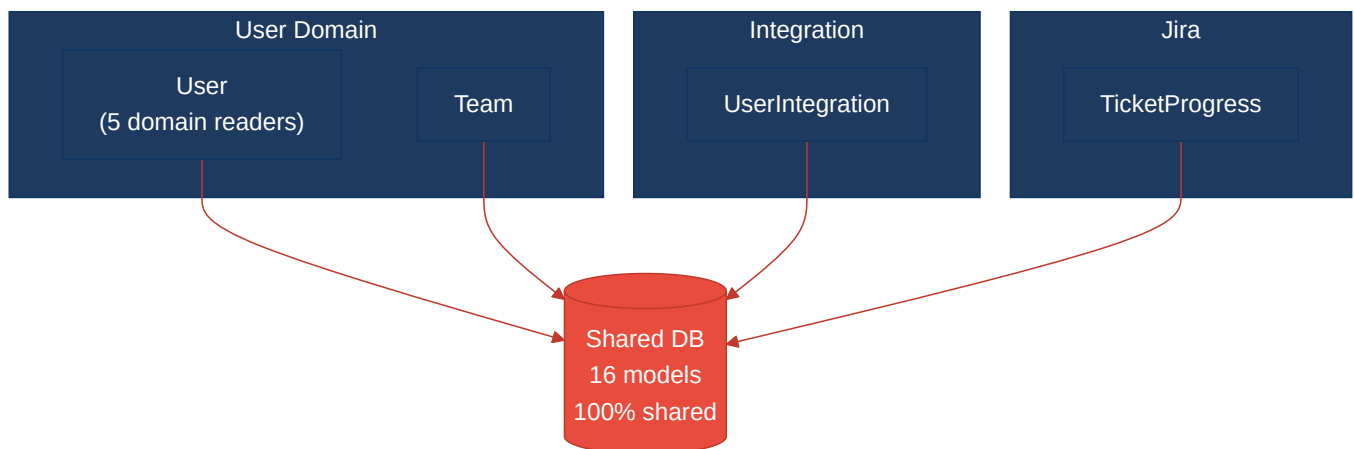
Current-state architecture (as-is)



### Frontend component architecture

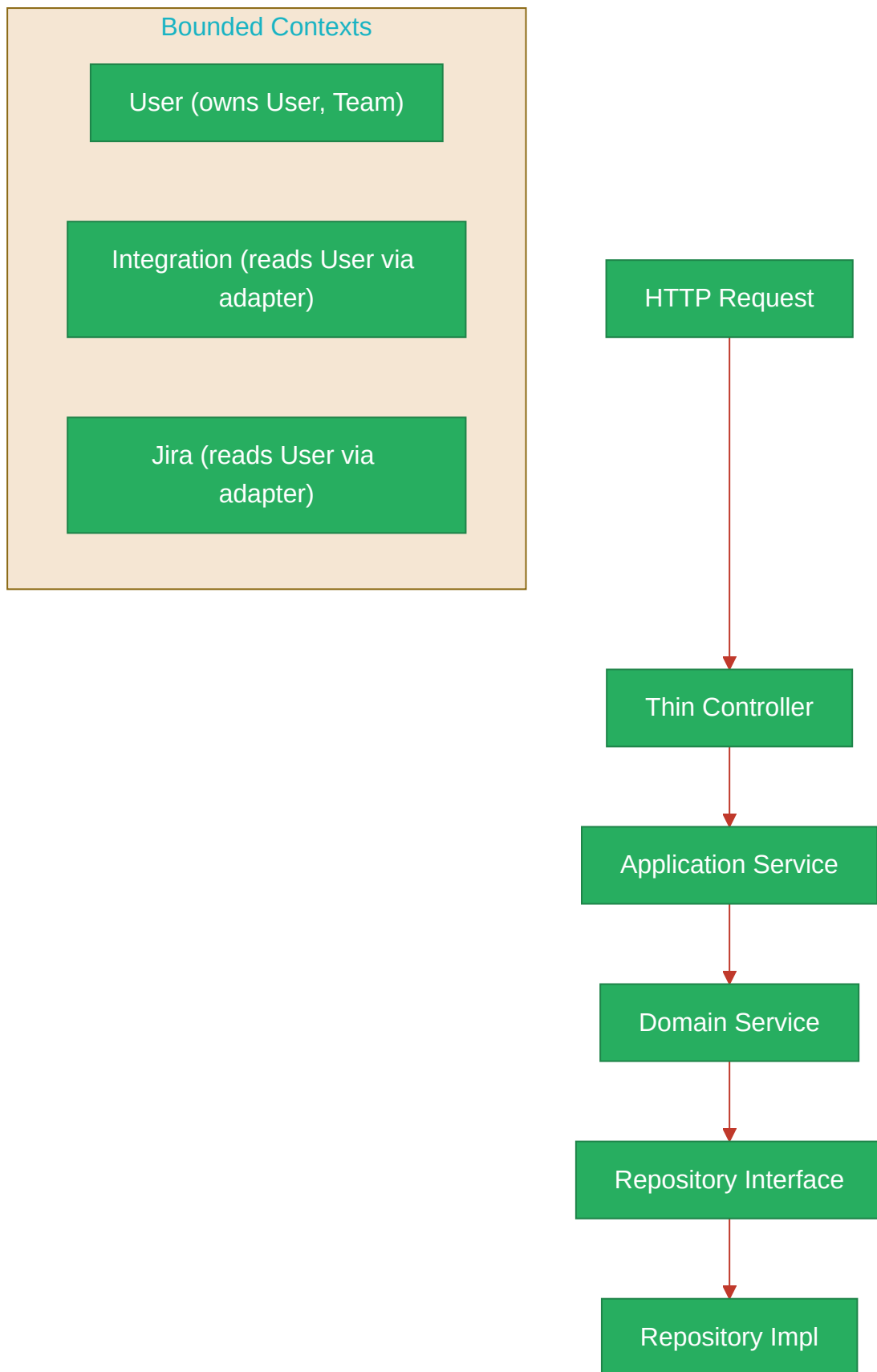


### Domain boundary map (current state)

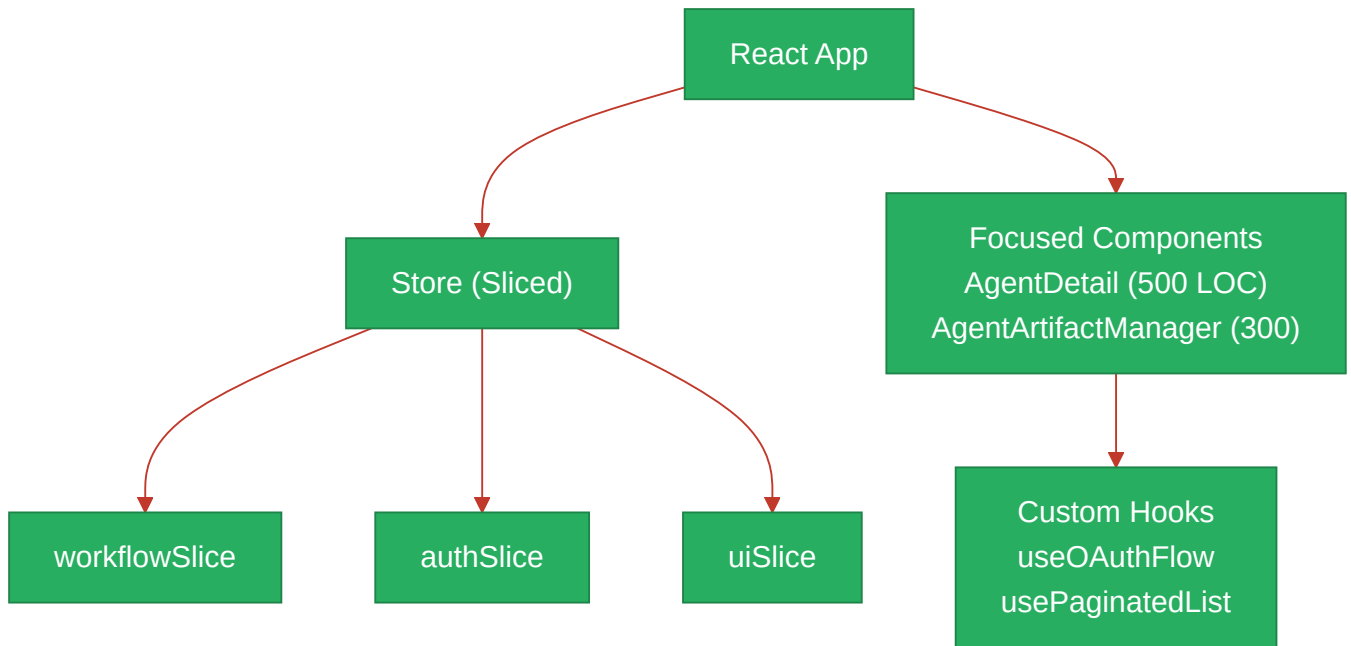


### Target architecture (proposed backend)

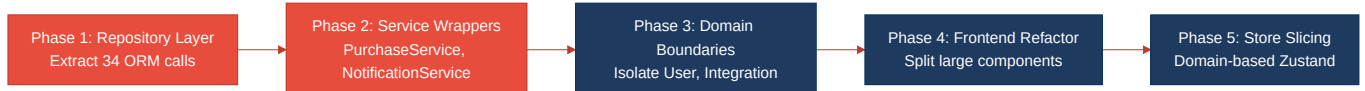




**Target architecture (proposed frontend)**



### Improvement roadmap



## 1.4 Actions Required

| Hotspot                                | Action                                                                                                                              | Rating    | Priority |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H3. Missing Repository Pattern         | Wrap 34 direct ORM calls into Repository classes. Inject via constructor DI. Target 100% ORM access through repositories.           | HIGH RISK | CRITICAL |
| H9. Shared Database Coupling           | Establish data ownership (User/Team domains are owners). Introduce anti-corruption layers for cross-domain reads (UserReadAdapter). | HIGH RISK | CRITICAL |
| F3. God / Oversized Components         | Split AgentDetail (2,453 LOC) into 5 focused components (~600 LOC each). Split Dashboard (1,338 LOC) into 3 components.             | HIGH RISK | CRITICAL |
| F1. Business Logic in Components       | Extract validation, filtering, calculations into utility functions and custom hooks. Remove business logic from render paths.       | HIGH RISK | CRITICAL |
| H2. Missing Service Layer              | Extract PurchaseService, NotificationService to wrap direct model access.                                                           | MODERATE  | HIGH     |
| H8. Domain Boundary Violations         | Replace direct User model imports in 4 services with userService.getUserById() calls.                                               | MODERATE  | HIGH     |
| F4. Prop Drilling / Global State Abuse | Slice useAppStore by domain. Use selector functions for targeted subscriptions. Implement React.memo on children.                   | HIGH RISK | HIGH     |
| H7. God Classes                        | Extract sub-modules from repoAstService, grafanaOAuthService, blueprintStore.                                                       | MODERATE  | MEDIUM   |

## 1.5 Expected Outcomes

---

- **Change Isolation & Testability:** Schema changes require updates in one place (repository), not cascading across 5 services. Business logic in custom hooks is unit-testable without rendering components.
- **Independent Scaling:** Each domain (User, Integration, Jira, Observability) can be independently tested, deployed, and extended without breaking consumers.
- **Developer Productivity:** Component refactoring reduces cognitive load. Custom hooks and utilities become discoverable, reducing copy-paste duplication.
- **Performance:** Store slicing and React.memo prevent unnecessary re-renders when workflow updates occur. Focused selector functions reduce subscriber count per store slice.
- **Architectural Clarity:** Bounded contexts, data ownership, and anti-corruption layers establish explicit boundaries between domains. New features no longer require modifying monolithic components or store.

---

**Report generated:** 2026-08-03 16:12:08 UTC | **Repository:** shende-shweta/pingcrm